

Software & Firmware Documentation: Smart Water Pump Controller

1. Technical Stack & Environment

The firmware is developed using the **Arduino Framework** for the **ESP32 DevKit V1 (38-pin)**. It leverages asynchronous, non-blocking logic to manage multiple safety-critical tasks simultaneously.

Key Libraries:

- `WiFi.h`: Core network connectivity.
- `Firebase_ESP_Client.h`: High-level wrapper for Firebase Real-Time Database (RTDB) interactions.
- `addons/TokenHelper.h` & `RTDBHelper.h`: Critical for secure authentication and payload parsing.

2. Hardware-Software Pin Mapping

The firmware is strictly mapped to the following GPIO pins, as defined in the system's mechanical and electrical blueprints:

Constant	GPIO	Type	Physical Component
RELAY_PIN	4	Output	5V Relay Module -> Contactor A1
TRIG_PIN	5	Output	JSN-SR04T Ultrasonic Trigger
ECHO_PIN	18	Input	JSN-SR04T Ultrasonic Echo (via 1kΩ/2kΩ divider)
FLOW_SENSOR_PIN	34	Input	YF-G1 Flow Sensor Signal (via 1kΩ/2kΩ divider)

3. Core Logic Engines

I. Ultrasonic Level Processing (`readUltrasonicSensor`)

The system measures distance in microseconds and converts it to a volume percentage for the 600L tank.

- **Calibration:**
 - TANK_EMPTY_CM (100cm): 0% capacity.
 - TANK_FULL_CM (15cm): 100% capacity.
- **Math:** Uses the `map()` function to normalize distance to a 0 – 100% scale.

II. Flow Rate Calculation (`calculateFlowRate`)

Uses a hardware interrupt on GPIO 34 to capture pulses in real-time without missing data.

- **Calibration:** $Q(L/min) \approx Pulses/1.0$.
- **Processing:** The pulse count is captured every 1,000ms, processed, and reset to zero.

III. State Machine (`executePumpLogic`)

The pump operates in three mutually exclusive states controlled via Firebase:

1. **AUTO:** Hysteresis-based automation.
 - **Start:** $Water \leq 30\%$.
 - **Stop:** $Water \geq 100\%$.
2. **FORCE_ON:** Direct manual override (Software-controlled).
3. **FORCE_OFF:** Direct manual shutdown (Software-controlled).

4. Safety & Protection Algorithms

Dry-Run Protection (`checkSafetyCutoff`)

This is a software-level failsafe that prevents the motor from running without water (which would overheat the pump head).

- **Threshold:** Flow Rate < 0.5 LPM.
- **Timer:** DRY_RUN_TIMEOUT_MS (30 seconds).
- **Action:** If the pump is active but flow is below the threshold for 30s, the system triggers `isDryRunError = true` and kills the relay.
- **Reset:** The error can only be cleared by a manual "Acknowledge" signal from the Web Dashboard.

5. Cloud & Network Integration

Firebase Data Structure

The system communicates with the Cloud via a flat JSON structure at the root `/pump_system/`.

Status Node (ESP32 → Cloud):

```
{  
  "water_level_percent": 85,  
  "is_running": true,
```

```
"flow_rate_lpm": 12.4,  
"is_error": false  
}
```

Control Node (Cloud → ESP32):

```
{  
  "mode": "AUTO",    // Options: AUTO, FORCE_ON, FORCE_OFF  
  "clear_error": false // Toggled by user to reset dry-run lockout  
}
```

Network Resiliency

- **Non-Blocking Sync:** Sensors are sampled every 1s, but Firebase is updated every 3s to prevent network congestion.
- **Autoreconnect:** `Firebase.reconnectWiFi(true)` ensures that if the house router reboots, the system recovers and re-syncs without human intervention.

6. Security Considerations

The firmware implements **Anonymous Authentication**. For the production phase, the `API_KEY` and `DATABASE_URL` should be restricted within the Firebase Console to only allow traffic from the specific IP address or domain of the Web Dashboard.